

SQL ADVANCED HANDS-ON

1. The CREATE TABLE statement is used to create a new table in a database. This statement defines the structure of the table by specifying the column names, data types, constraints, and other properties

Concept Implementation:

The borrow_book table includes columns for borrow details. It sets borrow_book_id as the primary key and establishes foreign key constraints to link with the user table respectively.

Description:

Write a query to create a borrow_book table with constraints at the table level.

Note: Assume that the user table is created already.

Borrow_book		
PK	borrow_book_id	varchar(25)
FK	user_id	varchar(10)
	borrow_date	date
	due_date	date
	return_date	date
	fine	decimal(25,2)

User		
PK	user_id	varchar(10)
	username	varchar(10)
	email	varchar(20)
	dob	Date
	gender	varchar(5)
	phoneno	varchar(10)
	address	varchar(20)

ANSWER:

```
1 CREATE TABLE Borrow_book(  
2 borrow_book_id varchar(25) ,  
3 user_id varchar(10),  
4 borrow_date date,  
5 due_date date,  
6 return_date date,  
7 fine decimal(25,2),  
8 PRIMARY KEY (borrow_book_id),  
9 FOREIGN KEY (user_id) REFERENCES User (user_id)  
0 );
```

2. Concept Implementation:

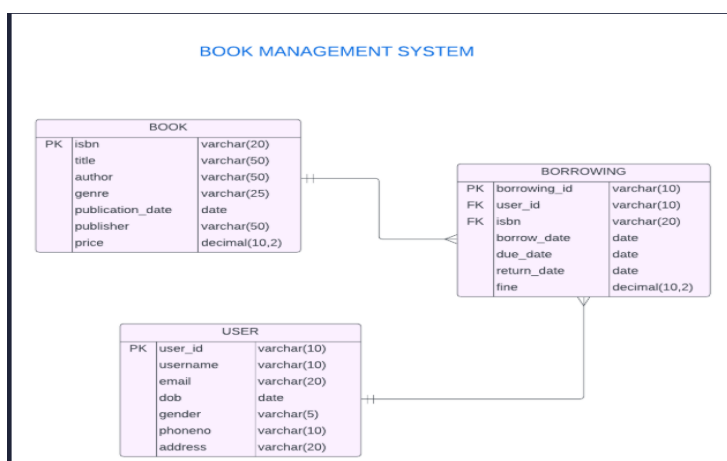
The ALTER TABLE table modifies the structure of the user table by renaming the column address to contact_info. This statement effectively updates the column name in the table schema without altering the data stored within it.

Description:

Refer to the given schema. Assume that the User table has already been created.

Write a query to rename a column **address** to **contact_info**.

(Note: Letters in bold represent the attribute name)



ANSWER:

```
ALTER TABLE USER CHANGE COLUMN address contact_info varchar(20);
```

3. Concept Implementation:

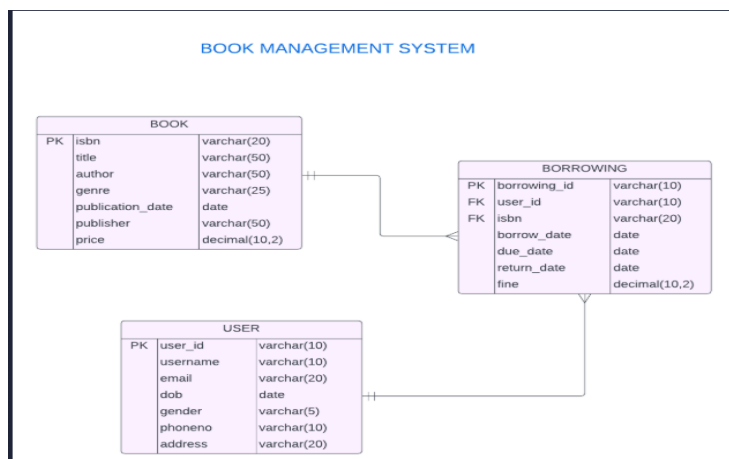
The ALTER TABLE table modifies the book table by changing the datatype size of the publisher column to VARCHAR(20) and adding a new column named no_of_books with the data type int to the book table, positioning it immediately after the publisher column.

Description:

Refer to the given schema. Assume that the book table has already been created.

Write a query to change the datatype size for **publisher** to varchar(20) and add a column **no_of_books** to int after the column publisher in the book table.

(Note: Letters in bold represent the attribute name)



ANSWER:

```
1 ALTER TABLE BOOK
2 CHANGE publisher publisher varchar(20),
3 ADD COLUMN no_of_books int AFTER publisher;
```

4. Concept Implementation:

The UPDATE statement to modify existing data in the Borrowing table's Return_Date column will be updated to '2023-07-13' and a fine of 15.00 is applied to the borrower's account.

Only the row where Borrowing_ID is 'BR006' will be affected by this update.

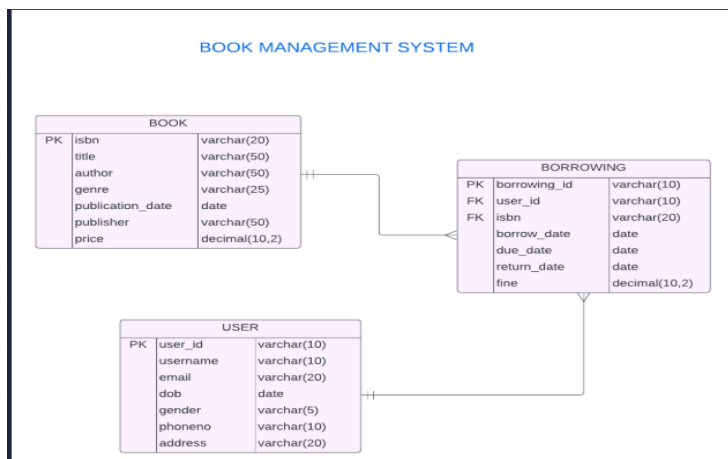
Description:

Refer to the given schema. Assume that the book table has already been created.

In a bustling library system, the librarian receives a notification regarding a specific borrowing record labelled 'BR006'. Upon inspection, they notice that the borrower has returned the item after the due date.

To maintain the system's integrity and ensure fairness, write a query to update the borrowing record, setting the return date to '2023-07-13' and applying a fine of 15.00 to the borrower's account.

(Hint: Data is case sensitive.)



ANSWER:

```
UPDATE BORROWING
SET return_date='2023-07-13', fine=15.00
where borrowing_id='BR006';
```

5. Concept Implementation:

The SELECT statement retrieves specific columns from the user table where the email does not contain the word yahoo. The results are then sorted in descending order based on the user_id column.

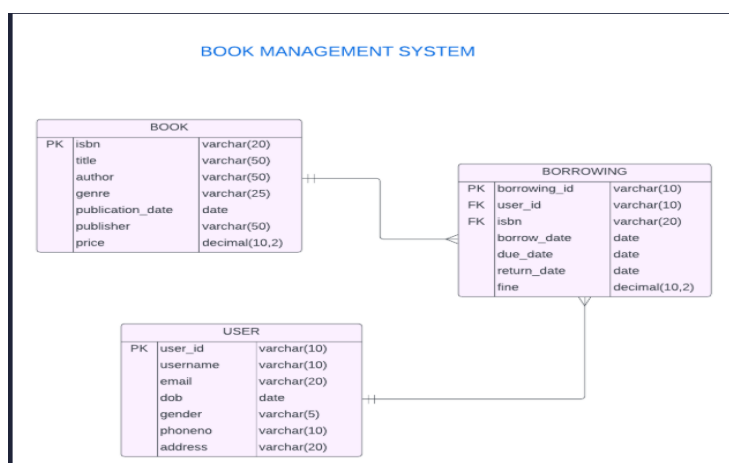
Description:

In the realm of digital marketing analytics, Claire, a data analyst for an e-commerce platform, is tasked with investigating user engagement patterns across different email service providers. She suspects that users without yahoo email addresses might exhibit distinct behaviours compared to users with yahoo email providers.

To gain deeper insights, write a SQL query to extract user information such as user id, name, and gender, specifically on those without yahoo email addresses.

Sort the records based on the user id in descending order.

Hint: Data is case-sensitive.



ANSWER:

```
SELECT user_id,username,gender FROM USER
WHERE email NOT LIKE '%@yahoo.com'
ORDER BY user_id DESC;
```

6. Concept Implementation:

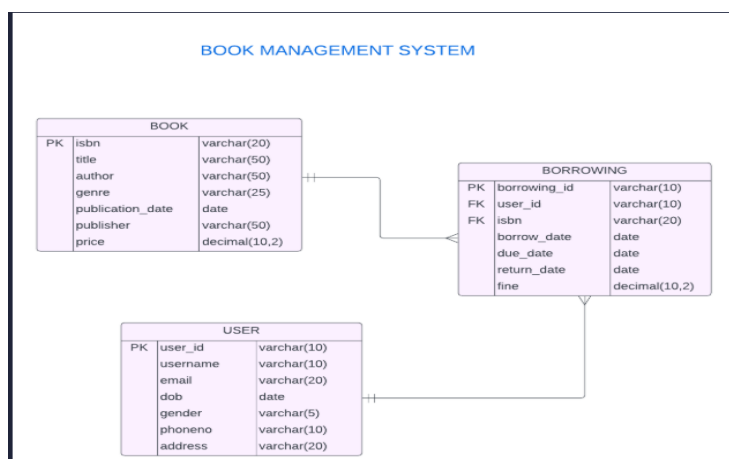
The SELECT statement retrieves specific columns from the book table, specifically selecting books with prices between 400 and 800. The results are then ordered by the price column in ascending order, listing books with lower prices first.

Description:

In a local bookstore, the manager is organising a promotional event showcasing a selection of books priced between 400 and 800.

Write a query to retrieve book details, including ISBN, title, genre, and price, filtering specifically for books within the designated price range.

Sort the records based on the price in ascending order.



ANSWER:

```
1 SELECT isbn,title,genre,price FROM BOOK
2 WHERE price BETWEEN 400 AND 800
3 ORDER BY price asc;
```

7. Concept Implementation:

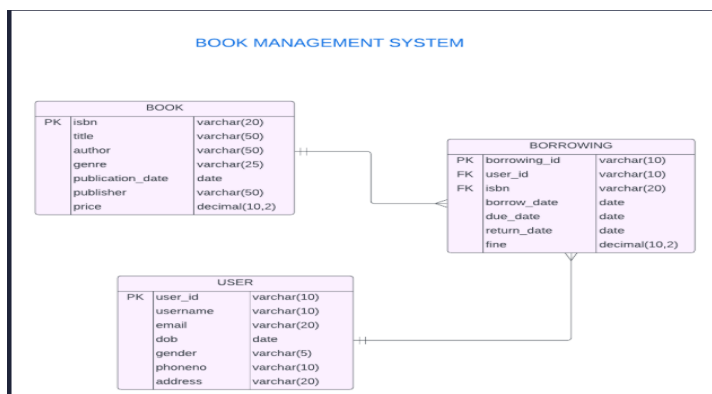
The SELECT statement retrieves specific columns from the book table for books that belong to either the 'Romance' or 'Mystery' genres. The results are then sorted in ascending order based on the title column.

Description:

Since the 'Romance' and 'Mystery' genres are the main focus of the next fair, Sarah wants to construct a query to select the ISBN, title, genre, and publisher from the database that fits into these categories.

Sort the records based on the title in ascending order.

Hint: Data is case-sensitive.



ANSWER:

```
1 SELECT isbn,title,genre,publisher FROM BOOK
2 WHERE genre="Romance" OR genre="Mystery"
3 ORDER BY title ASC;
```

8. Concept Implementation:

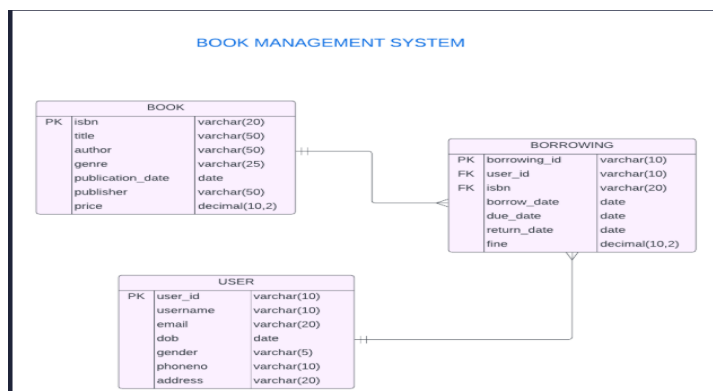
The SELECT statement retrieves specific columns from the borrowing table for records where the return_date is not the same as due_date. The results are then sorted in descending order based on the user_id, which represents the user's unique identifiers.

Description:

In a bustling library system, the librarian is conducting routine checks on borrowed items to ensure timely returns. Using a SQL query, they identify instances where the return date is not the same as the due date, indicating non-punctual or early returns by the borrowers.

Write a query to retrieve details such as isbn, user ID, and return date for these transactions.

Sort the records based on the user id in descending order.



ANSWER:

```
1 SELECT isbn,user_id,return_date FROM BORROWING
2 where return_date NOT LIKE due_date
3 ORDER BY user_id DESC;
```


9. Concept Implementation:

The SELECT statement retrieves specific columns from the user table. It takes the first three characters of the phone column and the first three characters of the username column, concatenates them, and assigns the result to the password column in the output. The results are then sorted in descending order based on the user_id.

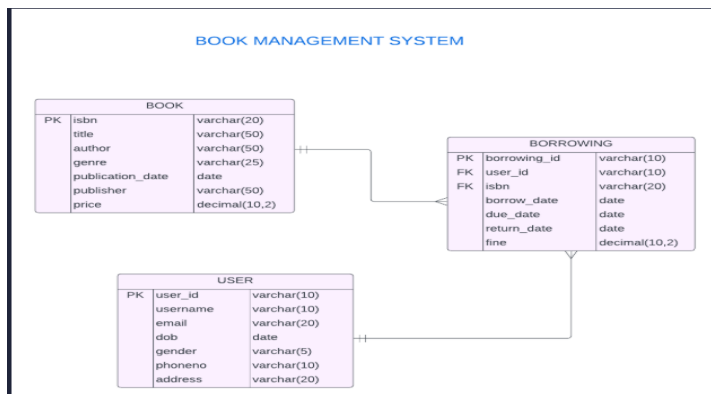
Description:

In a cybersecurity workshop, Ethan, the database administrator, is tasked with enhancing password security for user accounts on their platform. Ethan aims to generate and implement stronger passwords for all users, bolstering the platform's security measures and safeguarding user accounts against potential breaches.

To streamline the process, write a SQL query to select the user id, user name, and password for all the users.

The password is generated by concatenating the first three characters of the phone number and the first three characters of the username.

Sort the records based on the user id in descending order.



ANSWER:

```
1 SELECT user_id,username, CONCAT(left(phoneno,3),left(username,3)) AS password
2 FROM user
3 ORDER BY user_id DESC;
```

10. Concept Implementation:

The SELECT statement retrieves the user_id, username, and phone number or email details of the user, if both phone number and email are NULL then display as 'Contact not available', and alias it as contact from the user table. It further filters the results to include only records where the month of the dob is 'August'. The results are then sorted in ascending order based on the username.

Description:

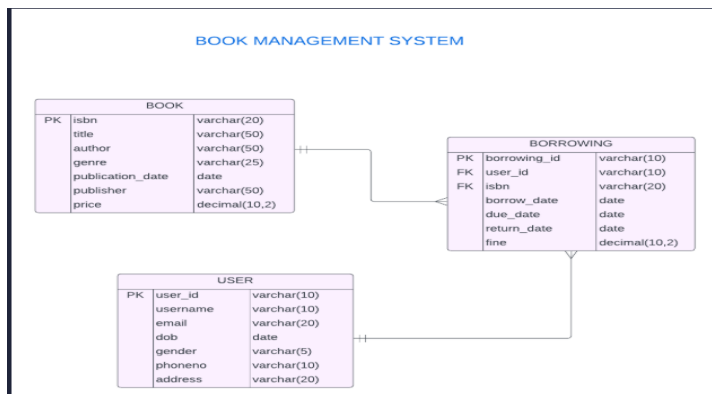
In anticipation of a springtime promotion, a marketing team at an online platform sifts through their user database for August birthdays.

Write a query to display the user ID, username, and phone number of all the users who were born in August. If the phone number is not available, display their email id, if the email is also not available, display 'Contact not available'.

Give an alias to the phone number as 'contact'.

Sort the records based on the username in ascending order.

Hint: Data is case-sensitive.



ANSWER:

```
1 select user_id,username,  
2 coalesce(phoneno,email,'Contact not available') AS contact from user  
3 where MONTHNAME(dob) ='August'  
4 order by username asc;
```

11. Concept Implementation:

The SELECT statement retrieves the user_id, username, dob, and calculates the age in years using the date function. It retrieves this information from the user table, specifically for users whose age other than 35 years. The results are then sorted in descending order based on the user id.

Description:

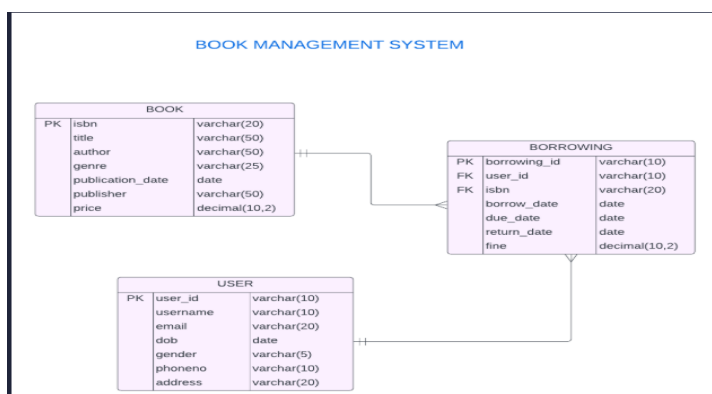
Within a thriving online community platform, administrators are keen to understand the demographic distribution of their users.

Write a SQL query to display the user ID, username, dob, and age of users whose does not belong to age 35.

Give an alias name to age as 'age_in_years'.

Sort the results based on the user id in descending order.

Note: Age must be rounded to the nearest whole number. Use datediff().



ANSWER:

```
1 select user_id,username,dob,  
2 ROUND(DATEDIFF(CURDATE(),DOB) / 365) AS age_in_years from USER  
3 where ROUND(DATEDIFF(CURDATE(),DOB) / 365) != 35  
4 order by user_id desc;  
5
```

12. Concept Implementation:

The SELECT statement concatenates the strings. It creates a string named `author_details` that contains information about the author and the publication year for each book in the BOOK table. The results are then sorted in ascending order based on the publication year.

Description:

Write an SQL query to display the author name and year of publication of all the books as given in the sample output.

Give an alias name as `author_details`.

The results should be sorted in ascending order based on the year of publication date.

Sample Data:

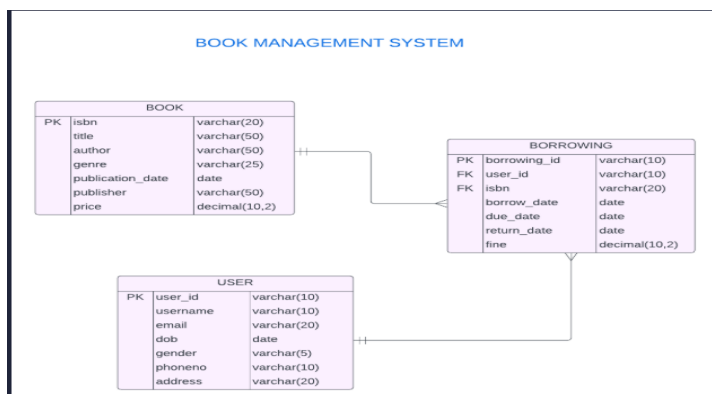
Mr/Ms.**author** published a book on the year **year of publication date**

Sample output:

`author_details`

Mr/Ms.John published a book on the year 2016

Hint: Data is case sensitive.



ANSWER:

```
1 select concat('Mr/Ms.',author,' published a book on the year '
2 ,YEAR(publication_date)) AS author_details
3 from book
4 order by YEAR(publication_date) asc;
```

13. Concept Implementation:

The SELECT statement retrieves the isbn, title, author and categorises books based on their price into three categories. The results are then sorted in ascending order based on the title column.

Description:

Write a query to display the isbn, title, author, and price_categorisation for all the books. price_categorisation is based on the following conditions:

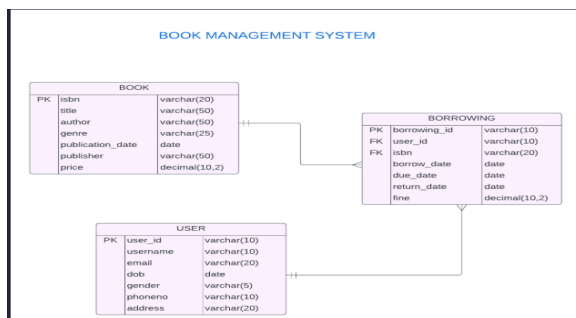
If the price is less than 300, then display 'Affordable Price',

If the price is greater than or equal to 300 and less than 500, then display 'Moderate Price'.

Otherwise, display 'Very Expensive'.

Sort the records based on the title in ascending order.

Hint: Data is case-sensitive.



ANSWER:

```
1 select isbn,title,author,
2 case
3 when price <300 then 'Affordable Price'
4 when price >=300 AND price <500 then 'Moderate Price'
5 else ' Very Expensive'
6 end as price_categorisation
7 from book
8 order by title asc;
```

14. Concept Implementation:

This Select statement retrieves user information from the user table and create an updated_email column that modifies the email by replacing the 'gmail.com' for users with 'Boston' addresses and retains the original email for others. The results are then ordered by user_id in descending order.

Description:

Users from the address 'Boston' have changed their email domain to 'gmail.com'.

Write a query to display the user id, user name, and updated_email for all the customers. Give an alias name as 'updated_email' for all the customers.

If the address is 'Boston' then change the email domain to 'gmail.com'.

Otherwise, display the email.

Sort the results based on the user id in descending order.

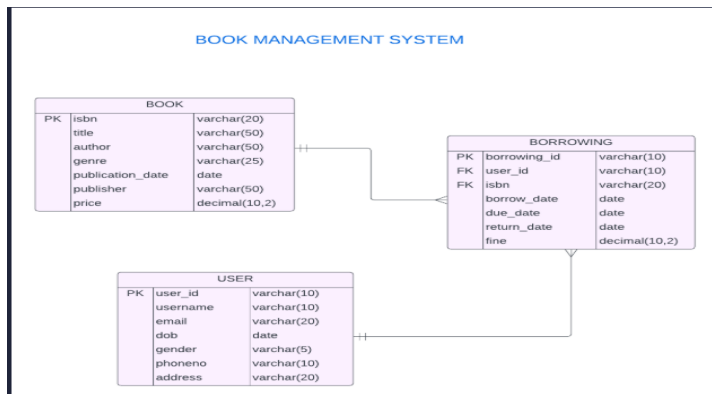
(Hint: Data is case sensitive.)

Sample Data:

user_id	username	email	address
C878	Warner	warn91@yahoo.com	Boston
C132	Starc	starc32@gmail.com	Boston
C726	Stokes	stokes@hotmail.com	Chicago

Sample Output:

user_id	username	updated_email
C878	Warner	warn91@gmail.com
C726	Stokes	stokes@hotmail.com
C132	Starc	starc32@gmail.com



ANSWER:

```

1  select user_id,username,
2  case
3  when address = 'Boston'
4  then concat(substring_index(email,'@',1),'@gmail.com')
5  else email
6  end as updated_email
7  from user
8  order by user_id desc;
  
```

15. Concept Implementation:

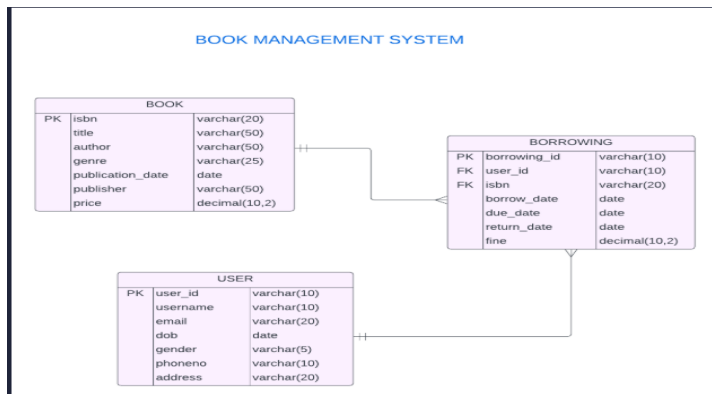
The query counts the number of books each user borrowed in the year 2024 by grouping user_id's. It uses the Scalar function to extract the year from borrow_date, filters records for the year 2024, calculates the count of borrowing IDs, and presents the result as count_book per user, sorting the output by user_id.

Description:

In 2024, the borrowing table was analyzed to count the number of books borrowed by each user.

Write a query to display the user id and count of books borrowed by each user during the year 2024. Give an alias name for the count of books as count_book.

The results should be sorted in descending order based on user id.



ANSWER:

```

1  select user_id,
2  count(borrowing_id) as count_book
3  from borrowing
4  where year(borrow_date) = 2024
5  group by user_id
6  order by user_id desc;
  
```

16. Concept Implementation:

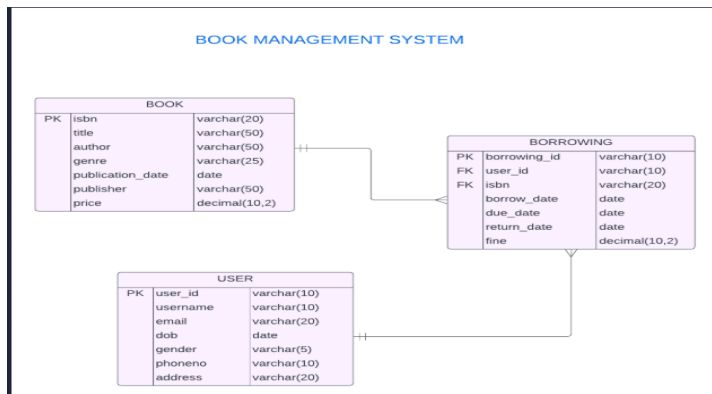
The query calculates the total number of books (books_total) written by a particular author in the book table by grouping records based on the author. It includes only authors who have written at least 1 book and orders the results in descending order based on the author.

Description:

In the vast library of literary wonders, there exists a realm where prolific authors reign supreme, their tales weaving intricate narratives that captivate readers far and wide. Among these storytellers, some have carved their names into the annals of history with a multitude of published works.

In order to identify such authors, you need to write an SQL query that displays the name of the authors who wrote at least one book and the number of books they have written. The total number of books is given an alias name of books_total.

The results should be sorted in descending order based on the author.



ANSWER:

```

1  select author,
2  case
3  when count(isbn) >= 1
4  then count(isbn)
5  end as books_total
6  from book
7  group by author
8  order by author desc;
  
```

17. Concept Explanation:

The joins are used to combine rows from two or more tables based on a related column between them. The concept of joins allows you to retrieve data that is spread across multiple tables in a database by linking records that have matching values in the specified columns.

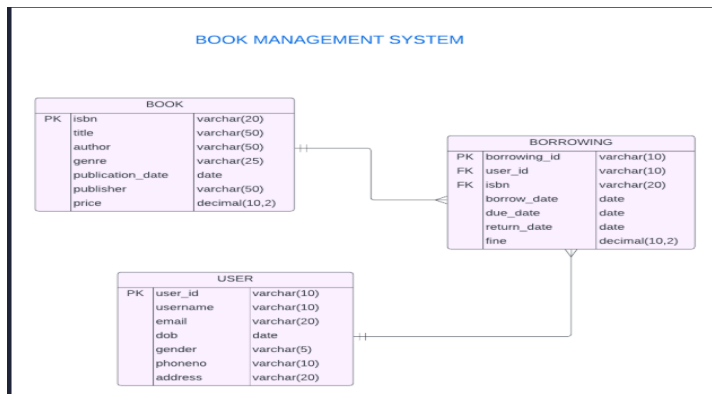
Concept Implementation:

The query retrieves the username, title, and due_date by joining data from the users, borrowing, and book tables based on matching user_id and isbn columns. The results are sorted in ascending order based on the due_date.

Description:

Write a query to display user name, title of the book borrowed by each user and due date to notify the book returning date.

The results should be sorted in ascending order based on the due date.



ANSWER:

```

1  select
2  u.username,
3  b.title,
4  br.due_date
5  from user u
6  join borrowing br on u.user_id=br.user_id
7  join book b on br.isbn= b.isbn
8  order by due_date asc;
  
```

18. Concept Implementation:

The query retrieves the username, borrowing_id, and formatted borrow_date as borrowed_date_info for each borrowing transaction, joining data from the users and borrowing tables based on matching columns. The Scalar function formats the date, and results are sorted in ascending order based on the borrowing id.

Description:

Write a query to display the user name, borrowing id and borrow_date of all the books.

Give an alias name for borrow_date as borrowed_date_info.

Sort the results based on the borrowing id ascending order.

The borrowed_date_info should be in the following format:

Eg: MonthName_Year

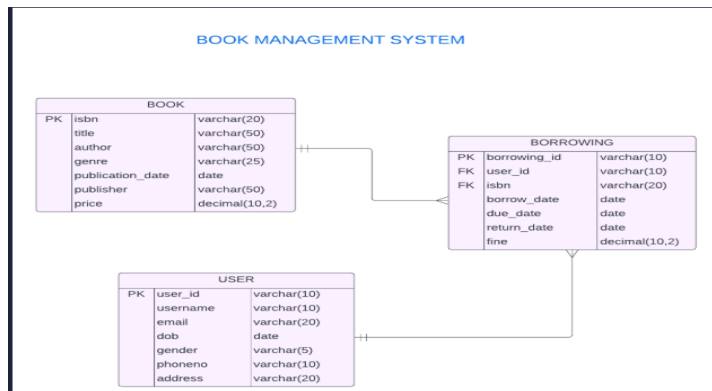
April_2016

Sample Output:

username borrowing_id borrowed_date_info

Sharma BK1 April_2017

Neha BK2 April_2018



ANSWER:

```
1 select
2 u.username,
3 br.borrowing_id,
4 DATE_FORMAT(br.borrow_date, '%M_%Y') AS borrowed_date_info
5 from user u
6 join borrowing br on u.user_id=br.user_id
7 order by borrowing_id asc;
```

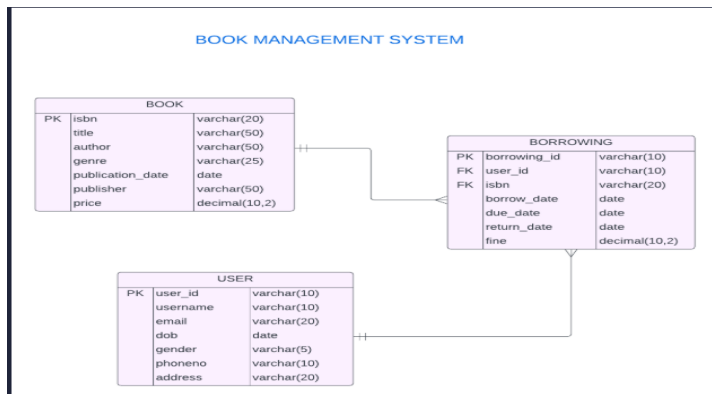
19. Concept Implementation:

The query retrieves data from the users, borrowing, and book tables based on matching **user_id** and **isbn** columns. It selects specific columns from these tables, filters records of users who have not returned the book yet and orders the results based on the user id.

Description:

Write an SQL query that fetches details about users who have borrowed books from the library but have not returned, including their user ID, username, email, ISBN, author, borrowing date, and due date.

The results should be sorted in descending order based on user id.



ANSWER:

```

1  select
2  u.user_id,
3  u.username,
4  u.email,
5  b.isbn,
6  b.author,
7  br.borrow_date,
8  br.due_date
9  from user u
10 join borrowing br on u.user_id=br.user_id
11 join book b on br.isbn=b.isbn
12
13 where br.return_date is null
14 order by u.user_id desc;
  
```

20. Concept Implementation:

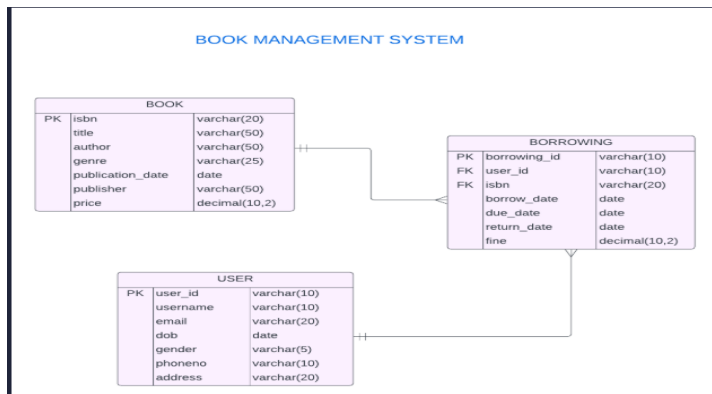
The query retrieves data from user information in the user table and calculates the minimum fine associated with each user by using a subquery from the borrowing table. If a user has no fines, the query returns null as the minimum fine. The results are then ordered based on the username in ascending order.

Description:

Write a SQL query that fetches details like username, phone number and the minimum fine amount paid by each user. Give an alias name for minimum fine as min_fine.

If the user has borrowed any book, then display the minimum fine amount; otherwise, display the minimum fine amount as null.

The results should be sorted in ascending order based on username.



ANSWER:

```

1  select
2  u.username,
3  u.phoneno,
4  (select min(fine)
5  from borrowing br
6  where br.user_id =u.user_id
7  ) as min_fine
8  from user u
9  order by u.username asc;|
  
```

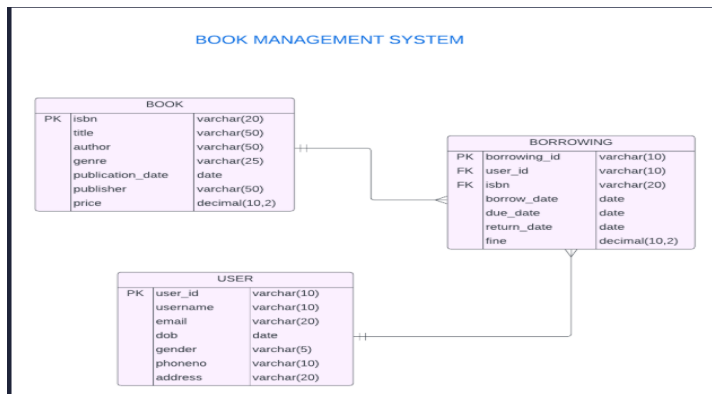
21. Concept Implementation:

The query retrieves distinct pairs of book titles (title1 and title2) along with their common author and genre. It joins the BOOK table with itself based on the condition that books have the same author. Additionally, it ensures that the books have the same genre but different titles. Finally, it orders the results in descending order based on the first book's title and in ascending order based on second book's title.

Description:

Write a query to display the title as title1, title as title2, author, and genre of all the books with the same author(s) who have written books in the same genre but with different titles.

Sort the records based on the title1 in descending order and title2 in ascending order.



ANSWER:

```

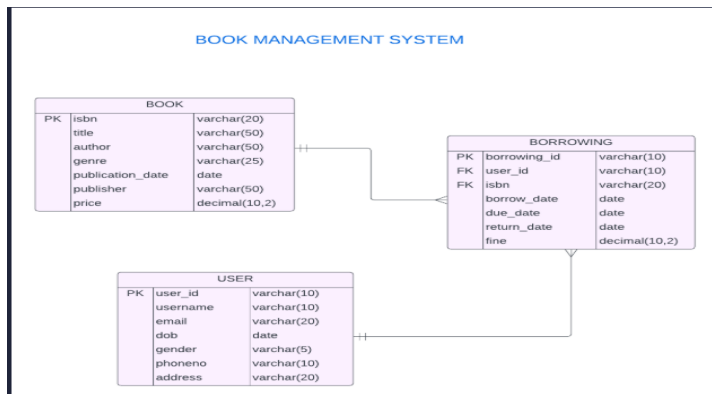
1  select
2  b1.title as title1,
3  b2.title as title2,
4  b1.author,
5  b1.genre
6  from book b1
7  join book b2
8  on b1.author = b2.author
9  and b1.genre = b2.genre
10 and b1.title != b2.title
11 order by b1.title desc,
12 b2.title asc;
  
```

22. Concept Implementation:

The query retrieves usernames from the user table for users who have borrowed books. It performs a join between the borrowing and user tables. The results are then grouped and filtered to include only users who have borrowed the minimum number of books compared to all other users, as determined by a subquery that calculates the count of borrowing IDs grouped by users. Order the results with respect to username in descending order.

Description:

Write a query to display the usernames of users who have borrowed the minimum number of books among all users. Sort the results based on username in descending order.



ANSWER:

```

1  select u.username
2  from user u
3  join borrowing b on u.user_id = b.user_id
4  group by u.username
5  having count(b.borrowing_id) =(
6  select min(book_count)
7  from(
8  select count(borrowing_id) as book_count
9  from borrowing
10 group by user_id
11 ) as subquery
12 )
13 order by u.username desc;
  
```

23. Concept Implementation:

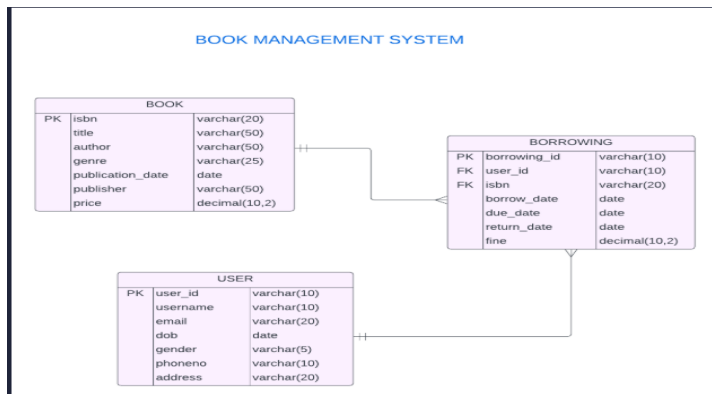
The query retrieves the username from the USER table and counts the number of borrowing IDs associated with each user. It uses a particular join that joins the BORROWING table based on matching user values. The results are then grouped and finally ordered in descending order based on user_id. This approach ensures that all users from the USERS table are included in the result, even if they have no borrowing records in the BORROWING table.

Description:

Write a query to display the username along with the number of books they have borrowed.

Give an alias name to the number of books as 'Number_of_books'.

Sort the records based on the user id in descending order.



ANSWER:

```

1  select u.username, count(b.borrowing_id) as 'Number_of_books'
2  from user u
3  left join borrowing b on u.user_id = b.user_id
4  group by u.user_id, u.username
5  order by u.user_id desc;
6

```

24. Concept Implementation:

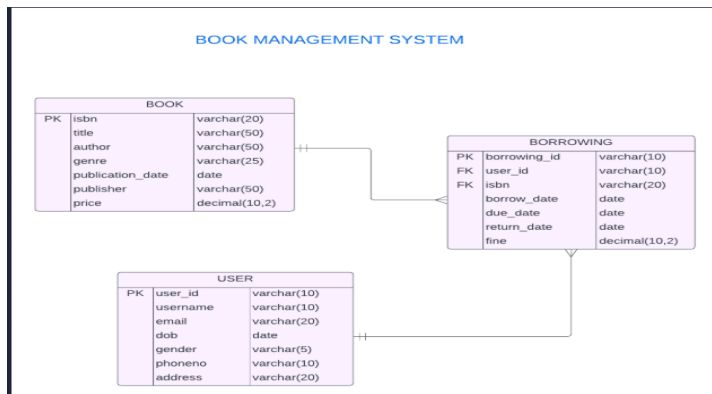
The query retrieves the author and price of books along with the count of user_id associated with each book. It joins the BOOK and BORROWING tables based on matching ISBN values and filters records to only include books that have been borrowed by a specific user ('UN006'). The results are then grouped by author and price, aggregating the count of unique users for each book.

Description:

Write an SQL query that retrieves information about a book borrowed by a specific user with user id **UN006**, including its author, price and the count of users who have borrowed the same book.

Give an alias name for the count of the users as count_user.

(Hint : Data is case-sensitive. Use Subquery)



ANSWER:

```
1 select b.author,b.price,count(br.user_id) as 'count_user'
2 from book b join borrowing br on b.isbn =br.isbn
3 where b.isbn in (
4 select isbn
5 from borrowing
6 where user_id='UN006'
7 )
8 group by b.author,b.price;
9
```

